Министерство Образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет инженерно-экономический
Кафедра экономической информатики

Дисциплина «Объектно-ориентированное проектирование и программирование»

ОТЧЕТ
по лабораторной работе №1
на тему
ИСПОЛЬЗОВАНИЕ ПОТОКОВ ВВОДА/ВЫВОДА ДАННЫХ
вариант №26

Студент гр. 572301 А.А. Левковский
Проверил ст.препод. каф. ЭИ М. Н. Салапура

Минск 2026

Цель: изучить понятие потока, организацию ввода данных из потока и вывода в поток, контроль состояния потока и исправление ошибок, неформатированный ввод-вывод, манипуляторы потоков (стандартные и определяемые пользователем).

ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Поток в C++ – это объект потокового класса, через который выполняется ввод или вывод данных; разные классы отвечают за разные типы потоков, например ifstream связан с файловым вводом. Использование потоковых классов вместо функций C (printf, scanf и т.п.) удобнее: каждый тип сам определяет, как его выводить, а операторы << и >> можно перегружать для своих классов, что упрощает работу и уменьшает число ошибок. Иерархия потоковых классов довольно сложна: базовым является ios_base, от него происходят basic_ios, затем basic_istream, basic_ostream и basic_iostream, а от них – файловые (ifstream, ofstream, fstream) и строковые потоки. Оператор извлечения >> объявлен в istream, оператор вставки << – в ostream; cout – стандартный выходной поток (экран), cin — стандартный входной поток (клавиатура), классы консольного ввода-вывода находятся в iostream, файловые – в fstream, манипуляторы – в iomanip. Класс ios_base хранит константы и флаги, общие для всех потоков, например showpoint и fixed; basic_istream содержит get, getline, read, basic_ostream – put, write, а basic_iostream наследует оба и подходит для чтения и записи.

Для управления форматом применяются флаги и манипуляторы: флаги устанавливаются методом setf() и снимаются unsetf(), действуют на все последующие операции до отмены, а манипуляторы включаются прямо в цепочку вывода и действуют только на последующие данные. Флаг можно задать как cout.setf(ios::flag), снять как cout.unsetf(ios::flag), а несколько флагов объединять через |. К основным флагам относятся: boolalpha и noboolalpha (вывод bool как true/false или 1/0), dec, oct, hex (системы счисления), showbase и noshowbase (показ основания), uppercase и nouppercase (регистр шестнадцатеричных цифр и показателя степени), showpos и noshowpos (явный знак +), scientific и fixed (научная или десятичная форма), internal (знак слева, число справа), showpoint и noshowpoint (показ дробной части), skipws и noskipws (пропуск или чтение пробелов), left и right (выравнивание по левому или правому краю). Манипуляторы бывают без аргументов (dec, hex, endl, fixed, left, right, showpoint и др.) и с аргументами (setw, setprecision, setfill, setiosflags, resetiosflags). Из iomanip доступны resetiosflags, setiosflags, setfill, setprecision, setw; из ios_base/basic_ios – fill(), precision(), width(), setf(), unsetf().

Двухаргументная версия setf(mask, unset) сбрасывает группу флагов и устанавливает нужный: например, cout.setf(ios::left, ios::adjustfield) сбрасывает все флаги выравнивания и включает left. Для групп определены basefield (dec, oct, hex), adjustfield (left, right, internal) и floatfield (scientific, fixed). Манипуляторы и функции позволяют настраивать точность, ширину поля,

символ заполнения и выравнивание, причём манипулятор действует только на следующий за ним элемент, а функция-флаг – на все последующие.

Класс basic_istream отвечает за извлечение данных: operator>> читает форматированные данные; get(Ch) читает один символ; get(str, count) и get(str, count, delim) читают в массив до перевода строки или до указанного разделителя; getline(str, count) и getline(str, count, delim) читают строку целиком до разделителя; putback(Ch) возвращает символ обратно в поток; ignore(count, delim) пропускает до count символов или до разделителя включительно; peek() возвращает следующий символ, не извлекая его; gcount() сообщает число символов, прочитанных при последнем неформатированном вводе; read(str, count) читает заданное число символов; seekg()/tellg() управляют позицией чтения. Класс basic_ostream отвечает за вставку: operator<< выводит форматированные данные; put(Ch) помещает один символ; flush() очищает буфер; write(str, count) пишет массив символов; seekp()/tellp() работают с позицией записи. Помимо cin и cout, есть cerr – небуферизованный поток ошибок (выводится сразу, не перенаправляется) и clog – буферизованный поток ошибок.

При ошибках ввода-вывода в потоке выставляются флаги состояния: badbit – потеря целостности буфера, eofbit – достигнут конец файла, failbit – не удалось извлечь допустимое поле, goodbit – ошибок нет. Проверить состояние можно методами eof(), fail(), good(), bad(), а сбросить флаги – методом clear() (без аргументов снимает все ошибки, с аргументом устанавливает указанный флаг). Типичная схема обработки: если cin.good() – ввод корректен, иначе cin.clear() сбрасывает ошибки, а cin.ignore(10, '\n') удаляет из потока лишние символы, после чего можно запросить ввод заново. Флаг failbit возникает, например, если вместо числа введены буквы или нажат Enter; при работе со строками он появляется при установленном noskipws, когда извлечение не может получить ни одного символа. Для корректной обработки ошибок важно вовремя очищать поток, иначе все последующие операции ввода будут неудачными.

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

26. Разработать набор классов (минимум 5 классов, связи между классами: агрегация, композиция, наследование) по предметной области «Станция технического обслуживания автомобилей». Функционал программы должен позволить собрать данные об услуге по обслуживанию автомобиля.

Листинг кода:

```
//l1_class_header.h
#pragma once
#include <iostream>
#include <string>
#include <fstream>

using namespace std;
```

```
class Service;

class CarOwner {
private:
    string full_name;
    string phone_number;
    string address;
    int car_number;

public:
    CarOwner();
    CarOwner(string full_name, string phone_number, string address, int car_number);
    void print_info(ostream& os);
    ~CarOwner();
};

class RepairOrder {
private:
    CarOwner owner;
    Service** services;
    string order_date;
    int order_id;
    double total_price;
    int service_count;
    int capacity;

public:
    RepairOrder();
    RepairOrder(CarOwner owner, string order_date, int order_id);
    ~RepairOrder();

    void add_service(Service* service);
    void remove_service(int index);
    void edit_service(int index, Service* new_service);
    double calculate_total();
    void print_receipt(ostream& os);
    bool save_to_file(const string& filename);
    void clear();

    int get_service_count() const { return service_count; }
    Service* get_service(int index) const { return services[index]; }
};

class Service {
protected:
    string name;
    string category;
    string duration;
    string complexity;
    double price;
    int slots;

public:
    Service();
    Service(string name, string category, string duration, string complexity, double
price, int slots);

    virtual void print_info();
    virtual string get_type() = 0;
    virtual ~Service();

    string get_name() const { return name; }
    string get_category() const { return category; }
    string get_duration() const { return duration; }
```

```
    string get_complexity() const { return complexity; }
    double get_price() const { return price; }
    int get_slots() const { return slots; }

    void set_name(const string& set) { name = set; }
    void set_category(const string& set) { category = set; }
    void set_duration(const string& set) { duration = set; }
    void set_complexity(const string& set) { complexity = set; }
    void set_price(double set) { price = set; }
    void set_slots(int set) { slots = set; }
};

class Diagnostics : public Service {
private:
    string equipment;

public:
    Diagnostics();
    Diagnostics(string name, string category, string duration, string complexity,
        double price, int slots, string equipment);

    void print_info() override;
    string get_type() override;
    ~Diagnostics();

    string get_equipment() const { return equipment; }
    void set_equipment(const string& v) { equipment = v; }
};

class Maintenance : public Service {
private:
    string interval;

public:
    Maintenance();
    Maintenance(string name, string category, string duration, string complexity,
        double price, int slots, string interval);

    void print_info() override;
    string get_type() override;
    ~Maintenance();

    string get_interval() const { return interval; }
    void set_interval(const string& v) { interval = v; }
};

class Repair : public Service {
private:
    string part_type;

public:
    Repair();
    Repair(string name, string category, string duration, string complexity,
        double price, int slots, string part_type);

    void print_info() override;
    string get_type() override;
    ~Repair();

    string get_part_type() const { return part_type; }
    void set_part_type(const string& v) { part_type = v; }
};

class ServiceStation {
private:
```

```
    string station_name;
    Service** catalog;
    RepairOrder** orders;
    int catalog_count;
    int catalog_capacity;
    int orders_count;
    int orders_capacity;

public:
    ServiceStation();
    ServiceStation(string station_name);
    ~ServiceStation();

    void add_to_catalog(Service* service);
    void remove_from_catalog(int index);
    void edit_catalog_item(int index, Service* new_service);

    void add_order(const CarOwner& owner);
    void find_item(const string& name);
    void show_catalog(ostream& os);
    void show_orders(ostream& os);
    void sort_catalog_by_price(bool sort_flag);
    bool save_all_to_file(const string& filename);

    void set_station_name(const string& name) { station_name = name; }

    int get_catalog_count() const { return catalog_count; }
    Service* get_catalog_item(int index) const { return catalog[index]; }

    int get_orders_count() const { return orders_count; }
    RepairOrder& get_order(int index) const { return *orders[index]; }
};

//lab1_classes.cpp
#include <iostream>
#include <fstream>
#include <iomanip>
#include <string>
#include <windows.h>
#include "l1_class_header.h"

using namespace std;

CarOwner::CarOwner() : full_name(""), phone_number(""), address(""), car_number(0)
{}

CarOwner::CarOwner(string full_name, string phone_number, string address, int
car_number)
    : full_name(full_name), phone_number(phone_number), address(address),
car_number(car_number) {
}

void CarOwner::print_info(ostream& os) {
    os << "Владелец: " << full_name
        << ", Телефон: " << phone_number
        << ", Адрес: " << address
        << ", Авто: " << car_number;
}

CarOwner::~CarOwner() {}

RepairOrder::RepairOrder()
    : owner(), services(nullptr), order_date(""), order_id(0),
    total_price(0), service_count(0), capacity(0) {
}
```

```cpp
RepairOrder::RepairOrder(CarOwner owner, string order_date, int order_id)
    : owner(owner), services(nullptr), order_date(order_date),
    order_id(order_id), total_price(0), service_count(0), capacity(0) {
}

RepairOrder::~RepairOrder() {
    delete[] services;
}

void RepairOrder::add_service(Service* service) {
    if (service_count >= capacity) {
        int new_cap = (capacity == 0) ? 4 : capacity * 2;
        Service** new_services = new Service * [new_cap];
        for (int i = 0; i < service_count; ++i) new_services[i] = services[i];
        delete[] services;
        services = new_services;
        capacity = new_cap;
    }
    services[service_count++] = service;
}

void RepairOrder::remove_service(int index) {
    if (index < 0 || index >= service_count) return;
    for (int i = index; i < service_count - 1; ++i) services[i] = services[i + 1];
    --service_count;
}

void RepairOrder::edit_service(int index, Service* new_service) {
    if (index < 0 || index >= service_count || !new_service) return;
    services[index] = new_service;
}

double RepairOrder::calculate_total() {
    total_price = 0;
    for (int i = 0; i < service_count; ++i)
        if (services[i]) total_price += services[i]->get_price();
    return total_price;
}

void RepairOrder::print_receipt(ostream& os) {
    os << "Заказ-наряд №" << order_id << " от " << order_date << "\n";
    owner.print_info(os);
    os << "\nУслуги:\n";

    os.setf(ios::fixed, ios::floatfield);
    os << setprecision(2);
    os.setf(ios::showpoint);
    os.setf(ios::right, ios::adjustfield);

    os << setw(71) << setfill('-') << "" << setfill(' ') << endl
        << setw(7) << "№" << " | "
        << setw(22) << "НАЗВАНИЕ" << " | "
        << setw(17) << "ТИП" << " | "
        << setw(14) << "ЦЕНА" << " | " << endl
        << setw(71) << setfill('-') << "" << setfill(' ') << endl;

    for (int i = 0; i < service_count; ++i) {
        os << setw(7) << i + 1 << " | "
            << setw(22) << services[i]->get_name() << " | "
            << setw(17) << services[i]->get_type() << " | "
            << setw(14) << services[i]->get_price() << " | " << endl
            << setw(71) << setfill('-') << "" << setfill(' ') << endl;
    }
```

```
    os << "Итого: " << calculate_total() << endl;

    os.unsetf(ios::fixed);
    os.unsetf(ios::showpoint);
    os.unsetf(ios::right);
}

bool RepairOrder::save_to_file(const string& filename) {
    ofstream out(filename);
    if (!out) return false;
    print_receipt(out);
    out.close();
    return true;
}

void RepairOrder::clear() {
    service_count = 0;
    total_price = 0;
}

Service::Service()
    : name(""), category(""), duration(""), complexity(""), price(0), slots(0) {
}

Service::Service(string name, string category, string duration, string complexity,
    double price, int slots)
    : name(name), category(category), duration(duration), complexity(complexity),
    price(price), slots(slots) {
}

void Service::print_info() {
    cout << "Название: " << name
        << ", Категория: " << category
        << ", Длительность: " << duration
        << ", Сложность: " << complexity
        << ", Цена: " << price
        << ", Мест: " << slots;
}

Service::~Service() {}

Diagnostics::Diagnostics() : Service(), equipment("") {}

Diagnostics::Diagnostics(string name, string category, string duration, string
complexity,
    double price, int slots, string equipment)
    : Service(name, category, duration, complexity, price, slots),
equipment(equipment) {
}

void Diagnostics::print_info() { Service::print_info(); cout << ", Оборудование: "
<< equipment; }
string Diagnostics::get_type() { return "Диагностика"; }
Diagnostics::~Diagnostics() {}

Maintenance::Maintenance() : Service(), interval("") {}

Maintenance::Maintenance(string name, string category, string duration, string
complexity,
    double price, int slots, string interval)
    : Service(name, category, duration, complexity, price, slots),
interval(interval) {
}
```

```
void Maintenance::print_info() { Service::print_info(); cout << ", Интервал: " <<
interval; }
string Maintenance::get_type() { return "ТО"; }
Maintenance::~Maintenance() {}

Repair::Repair() : Service(), part_type("") {}

Repair::Repair(string name, string category, string duration, string complexity,
    double price, int slots, string part_type)
    : Service(name, category, duration, complexity, price, slots),
part_type(part_type) {
}

void Repair::print_info() { Service::print_info(); cout << ", Деталь: " <<
part_type; }
string Repair::get_type() { return "Ремонт"; }
Repair::~Repair() {}

ServiceStation::ServiceStation()
    : station_name(""), catalog(nullptr), orders(nullptr),
    catalog_count(0), catalog_capacity(0),
    orders_count(0), orders_capacity(0) {
}

ServiceStation::ServiceStation(string station_name)
    : station_name(station_name), catalog(nullptr), orders(nullptr),
    catalog_count(0), catalog_capacity(0),
    orders_count(0), orders_capacity(0) {
}

ServiceStation::~ServiceStation() {
    for (int i = 0; i < catalog_count; ++i) delete catalog[i];
    delete[] catalog;
    for (int i = 0; i < orders_count; ++i) delete orders[i];
    delete[] orders;
}

void ServiceStation::add_to_catalog(Service* service) {
    if (catalog_count >= catalog_capacity) {
        int new_cap = (catalog_capacity == 0) ? 4 : catalog_capacity * 2;
        Service** new_catalog = new Service * [new_cap];
        for (int i = 0; i < catalog_count; ++i) new_catalog[i] = catalog[i];
        delete[] catalog;
        catalog = new_catalog;
        catalog_capacity = new_cap;
    }
    catalog[catalog_count++] = service;
}

void ServiceStation::remove_from_catalog(int index) {
    if (index < 0 || index >= catalog_count) return;
    delete catalog[index];
    for (int i = index; i < catalog_count - 1; ++i) catalog[i] = catalog[i + 1];
    --catalog_count;
}

void ServiceStation::edit_catalog_item(int index, Service* new_service) {
    if (index < 0 || index >= catalog_count || !new_service) return;
    delete catalog[index];
    catalog[index] = new_service;
}

void ServiceStation::add_order(const CarOwner& owner) {
    if (orders_count >= orders_capacity) {
        int new_cap = (orders_capacity == 0) ? 4 : orders_capacity * 2;
```

```
        RepairOrder** new_orders = new RepairOrder * [new_cap];
        for (int i = 0; i < orders_count; ++i) new_orders[i] = orders[i];
        delete[] orders;
        orders = new_orders;
        orders_capacity = new_cap;
    }
    string date;
    getline(cin, date);
    orders[orders_count] = new RepairOrder(owner, date, orders_count + 1);
    ++orders_count;
}

void ServiceStation::find_item(const string& name) {
    bool found = false;
    for (int i = 0; i < catalog_count; ++i) {
        if (catalog[i]->get_name() == name) {
            catalog[i]->print_info();
            cout << endl;
            found = true;
        }
    }
    if (!found) cout << "Услуга не найдена.\n";
}

void ServiceStation::show_catalog(ostream& os) {
    os.setf(ios::fixed, ios::floatfield);
    os << setprecision(2);
    os.setf(ios::showpoint);
    os.setf(ios::right, ios::adjustfield);

    os << setw(120) << setfill('-') << "" << setfill(' ') << endl
        << setw(8) << "№ | "
        << setw(23) << "НАЗВАНИЕ | "
        << setw(18) << "КАТЕГОРИЯ | "
        << setw(13) << "ДЛИТ. | "
        << setw(13) << "СЛОЖН. | "
        << setw(15) << "ЦЕНА | "
        << setw(14) << "МЕСТ | "
        << setw(16) << "ТИП |" << endl
        << setw(120) << setfill('-') << "" << setfill(' ') << endl;

    for (int i = 0; i < catalog_count; ++i) {
        os << setw(5) << i + 1 << " | "
            << setw(20) << catalog[i]->get_name() << " | "
            << setw(15) << catalog[i]->get_category() << " | "
            << setw(10) << catalog[i]->get_duration() << " | "
            << setw(10) << catalog[i]->get_complexity() << " | "
            << setw(12) << catalog[i]->get_price() << " | "
            << setw(10) << catalog[i]->get_slots() << " | "
            << setw(15) << catalog[i]->get_type() << " |" << endl
            << setw(120) << setfill('-') << "" << setfill(' ') << endl;
    }

    os.unsetf(ios::fixed);
    os.unsetf(ios::showpoint);
    os.unsetf(ios::right);
}

void ServiceStation::show_orders(ostream& os) {
    if (orders_count == 0) { os << "Заказ-нарядов нет.\n"; return; }
    for (int i = 0; i < orders_count; ++i) {
        orders[i]->print_receipt(os);
        os << "--------------------\n";
    }
}
```

```
bool ServiceStation::save_all_to_file(const string& filename) {
    ofstream out(filename);
    if (!out) return false;
    out << "СТО: " << station_name << "\n\n";
    out << "КАТАЛОГ УСЛУГ:\n";
    show_catalog(out);
    out << "\nЗАКАЗ-НАРЯДЫ:\n";
    show_orders(out);
    out.close();

    ifstream in(filename);
    if (!in) return false;
    string line;
    while (getline(in, line)) {}
    if (in.bad()) cout << "Проверка потока: bad.\n";
    else if (in.eof()) cout << "Проверка потока: eof.\n";
    else if (in.fail()) cout << "Проверка потока: fail.\n";
    else if (in.good()) cout << "Проверка потока: good.\n";
    in.clear();
    return true;
}

//lab1_main.cpp
#include <iostream>
#include <fstream>
#include <iomanip>
#include <windows.h>
#include "l1_class_header.h"
#define FILE_NAME "lab1_file.txt"

using namespace std;

Service* create_service_from_input() {
    int type;
    cout << "Тип: 1-Диагностика 2-ТО 3-Ремонт: ";
    cin >> type;
    cin.ignore(10000, '\n');

    string name, category, duration, complexity, extra;
    double price;
    int slots;

    cout << "Название: ";
    getline(cin, name);
    cout << "Категория: ";
    getline(cin, category);
    cout << "Длительность: ";
    getline(cin, duration);
    cout << "Сложность: ";
    getline(cin, complexity);
    cout << "Цена: ";
    cin >> price;
    cout << "Количество мест: ";
    cin >> slots;
    cin.ignore(10000, '\n');

    if (type == 1) {
        cout << "Оборудование: "; getline(cin, extra);
        return new Diagnostics(name, category, duration, complexity, price, slots,
extra);
    }
    else if (type == 2) {
        cout << "Интервал: "; getline(cin, extra);
```

```
        return new Maintenance(name, category, duration, complexity, price, slots,
extra);
    }
    else {
        cout << "Тип детали: "; getline(cin, extra);
        return new Repair(name, category, duration, complexity, price, slots,
extra);
    }
}

void edit_service_fields(Service* item) {
    cout << "Что редактировать?\n";
    cout << "1. Название\n2. Категория\n3. Длительность\n4. Сложность\n5. Цену\n6.
Количество мест\n";
    cout << "0. Отмена\n";
    cout << "Выбор: ";
    int c; cin >> c; cin.ignore(10000, '\n');

    string s; double p; int q;
    switch (c) {
    case 1: cout << "Новое название: ";
        getline(cin, s); item->set_name(s); break;
    case 2: cout << "Новая категория: ";
        getline(cin, s); item->set_category(s); break;
    case 3: cout << "Новая длительность: ";
        getline(cin, s); item->set_duration(s); break;
    case 4: cout << "Новая сложность: ";
        getline(cin, s); item->set_complexity(s); break;
    case 5: cout << "Новая цена: ";
        cin >> p;
        cin.ignore(10000, '\n'); item->set_price(p); break;
    case 6: cout << "Новое кол-во мест: ";
        cin >> q;
        cin.ignore(10000, '\n'); item->set_slots(q); break;
    default:
        cout << "Отмена.\n";
        break;
    }
}

int main() {
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);

    ServiceStation station("СТО АвтоМастер");

    station.add_to_catalog(new Diagnostics("Комп. диагностика", "Электрика", "30
мин", "Средняя", 45, 3, "Сканер OBD-II"));
    station.add_to_catalog(new Maintenance("Замена масла", "Двигатель", "40 мин",
"Низкая", 75, 5, "каждые 10 000 км"));
    station.add_to_catalog(new Repair("Замена колодок", "Тормозная сист.", "1 час",
"Средняя", 125, 4, "колодки"));

    int choice = 0;

    do {
        system("cls");
        cout << "==========СТО: АвтоМастер==========\n";
        cout << "1. Оформить заказ-наряд\n";
        cout << "2. Просмотреть каталог услуг\n";
        cout << "3. Просмотреть информацию о заказ-нарядах\n";
        cout << "4. Изменить название СТО\n";
        cout << "5. Управление каталогом услуг\n";
        cout << "6. Управление заказ-нарядом\n";
        cout << "0. Сохранить и выйти\n";
```

```
        cout << "====================================\n\n";
        cout << "Ваш выбор: ";

        cin >> choice;
        if (cin.fail()) { cin.clear(); cin.ignore(10000, '\n'); choice = -1; }

        switch (choice) {
        case 1: {
            system("cls");
            cin.ignore(10000, '\n');

            string name, phone, address;
            int car_number;

            cout << "Введите ФИО владельца: ";
            getline(cin, name);
            cout << "Введите телефон: ";
            getline(cin, phone);
            cout << "Введите адрес: ";
            getline(cin, address);
            cout << "Введите номер автомобиля: ";
            cin >> car_number;

            CarOwner owner(name, phone, address, car_number);
            station.add_order(owner);

            int order_idx = station.get_orders_count() - 1;
            RepairOrder& order = station.get_order(order_idx);

            char more = 'y';
            while (more == 'y' || more == 'Y') {
                station.show_catalog(cout);
                cout << "Выберите номер услуги (Выход - 0): ";
                int item_idx; cin >> item_idx;

                if (item_idx > 0 && item_idx <= station.get_catalog_count() &&
station.get_catalog_item(item_idx - 1)->get_slots() > 0) {
                    order.add_service(station.get_catalog_item(item_idx - 1));
                    station.get_catalog_item(item_idx - 1)-
>set_slots(station.get_catalog_item(item_idx - 1)->get_slots() - 1);
                    cout << "Услуга добавлена.\n";
                }

                else if (item_idx == 0) {
                    break;
                }

                else if (station.get_catalog_item(item_idx - 1)->get_slots() == 0) {
                    cout << "Свободных мест нет.\n";
                }

                else {
                    cout << "Неверный номер.\n";
                }

                cout << "Добавить ещё? (y/n): ";
                cin >> more;
            }

            order.calculate_total();
            if (order.calculate_total() > 0) {
                cout.setf(ios::fixed, ios::floatfield);
                cout << setprecision(2) << "Заказ-наряд оформлен. Итоговая сумма: "
                    << order.calculate_total() << endl;
                cout.unsetf(ios::fixed);
```

```
            }
            else cout << "Заказ-наряд пуст. Отмена";

            system("pause");
            break;
        }

        case 2: {
            system("cls");
            station.show_catalog(cout);
            system("pause");
            break;
        }

        case 3: {
            system("cls");
            station.show_orders(cout);
            system("pause");
            break;
        }

        case 4: {
            system("cls");
            cin.ignore(10000, '\n');
            string stationName;
            cout << "Введите название СТО: ";
            getline(cin, stationName);
            station.set_station_name(stationName);
            cout << "Название СТО обновлено.\n";
            system("pause");
            break;
        }

        case 5: {
            int sub = 0;
            do {
                system("cls");
                cout << "---Управление каталогом услуг---\n";
                cout << "1. Добавить услугу\n";
                cout << "2. Редактировать услугу\n";
                cout << "3. Удалить услугу\n";
                cout << "0. Назад\n";
                cout << "Выбор: ";
                cin >> sub;
                if (cin.fail()) { cin.clear(); cin.ignore(10000, '\n'); sub = -1; }

                if (sub == 1) {
                    system("cls");
                    cin.ignore(10000, '\n');
                    Service* item = create_service_from_input();
                    station.add_to_catalog(item);
                    cout << "Услуга добавлена в каталог.\n";
                    system("pause");
                }
                else if (sub == 2) {
                    system("cls");
                    station.show_catalog(cout);
                    if (station.get_catalog_count() == 0) { system("pause");
continue; }
                    cout << "Введите номер услуги для редактирования: ";
                    int idx; cin >> idx;
                    cin.ignore(10000, '\n');
                    if (idx > 0 && idx <= station.get_catalog_count()) {
                        edit_service_fields(station.get_catalog_item(idx - 1));
                        cout << "Услуга изменена.\n";
```

```
                    }
                    else {
                        cout << "Неверный номер.\n";
                    }
                    system("pause");
                }
                else if (sub == 3) {
                    system("cls");
                    station.show_catalog(cout);
                    if (station.get_catalog_count() == 0) { system("pause");
continue; }
                    cout << "Введите номер услуги для удаления: ";
                    int idx; cin >> idx;
                    if (idx > 0 && idx <= station.get_catalog_count()) {
                        station.remove_from_catalog(idx - 1);
                        cout << "Услуга удалена.\n";
                    }
                    else {
                        cout << "Неверный номер.\n";
                    }
                    system("pause");
                }
            } while (sub != 0);
            break;
        }

        case 6: {
            system("cls");
            if (station.get_orders_count() == 0) {
                cout << "Заказ-нарядов нет.\n";
                system("pause");
                break;
            }
            station.show_orders(cout);
            cout << "Введите номер заказ-наряда: ";
            int order_idx; cin >> order_idx;
            if (order_idx < 1 || order_idx > station.get_orders_count()) {
                cout << "Неверный номер.\n";
                system("pause");
                break;
            }
            RepairOrder& order = station.get_order(order_idx - 1);

            int sub = 0;
            do {
                system("cls");
                cout << "---Управление заказ-нарядом №" << order_idx << "---\n";
                order.print_receipt(cout);
                cout << "\n1. Добавить услугу в заказ-наряд\n";
                cout << "2. Редактировать услугу в заказ-наряде\n";
                cout << "3. Удалить услугу из заказ-наряда\n";
                cout << "0. Назад\n";
                cout << "Выбор: ";
                cin >> sub;
                if (cin.fail()) { cin.clear(); cin.ignore(10000, '\n'); sub = -1; }

                if (sub == 1) {
                    system("cls");
                    station.show_catalog(cout);
                    if (station.get_catalog_count() == 0) { system("pause");
continue; }
                    cout << "Введите номер услуги: ";
                    int idx; cin >> idx;
                    if (idx > 0 && idx <= station.get_catalog_count()) {
                        order.add_service(station.get_catalog_item(idx - 1));
```

```
                        order.calculate_total();
                        cout << "Услуга добавлена в заказ-наряд.\n";
                    }
                    else {
                        cout << "Неверный номер.\n";
                    }
                    system("pause");
                }
                else if (sub == 2) {
                    system("cls");
                    order.print_receipt(cout);
                    if (order.get_service_count() == 0) { system("pause"); continue;
}
                    cout << "Введите номер позиции заказ-наряда для редактирования:
";
                    int idx; cin >> idx;
                    if (idx > 0 && idx <= order.get_service_count()) {
                        station.show_catalog(cout);
                        cout << "Введите номер новой услуги из каталога: ";
                        int new_idx; cin >> new_idx;
                        if (new_idx > 0 && new_idx <= station.get_catalog_count()) {
                            order.edit_service(idx - 1,
station.get_catalog_item(new_idx - 1));
                            order.calculate_total();
                            cout << "Позиция заказ-наряда обновлена.\n";
                        }
                        else {
                            cout << "Неверный номер услуги.\n";
                        }
                    }
                    else {
                        cout << "Неверная позиция.\n";
                    }
                    system("pause");
                }
                else if (sub == 3) {
                    system("cls");
                    order.print_receipt(cout);
                    if (order.get_service_count() == 0) { system("pause"); continue;
}
                    cout << "Введите номер позиции для удаления: ";
                    int idx; cin >> idx;
                    if (idx > 0 && idx <= order.get_service_count()) {
                        order.remove_service(idx - 1);
                        order.calculate_total();
                        cout << "Позиция удалена.\n";
                    }
                    else {
                        cout << "Неверный номер.\n";
                    }
                    system("pause");
                }
            } while (sub != 0);
            break;
        }

        case 0: {
            system("cls");
            if (station.save_all_to_file(FILE_NAME))
                cout << "Данные сохранены в " << FILE_NAME << endl;
            else
                cout << "Ошибка сохранения.\n";
            break;
        }
```

```
        default:
            cout << "Неверный выбор" << endl;
            system("pause");
            break;
        }
    } while (choice != 0);

    cout << "До свидания" << endl;
    return 0;
}
```

Примеры работы программы приведены на русунках 1, 2, 3, 4. На рисунке 5 изображён пример вывода данных в текстовый файл. UML диаграмма созданных классов показана на рисунке 6.

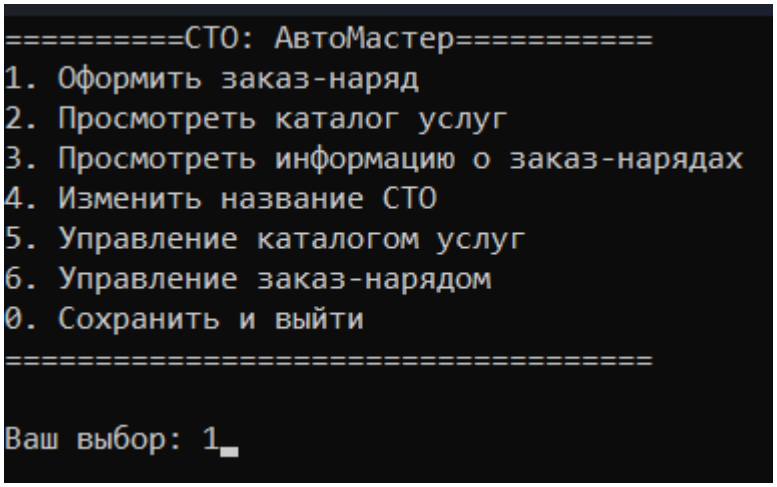


Рисунок 1 – Пример работы программы

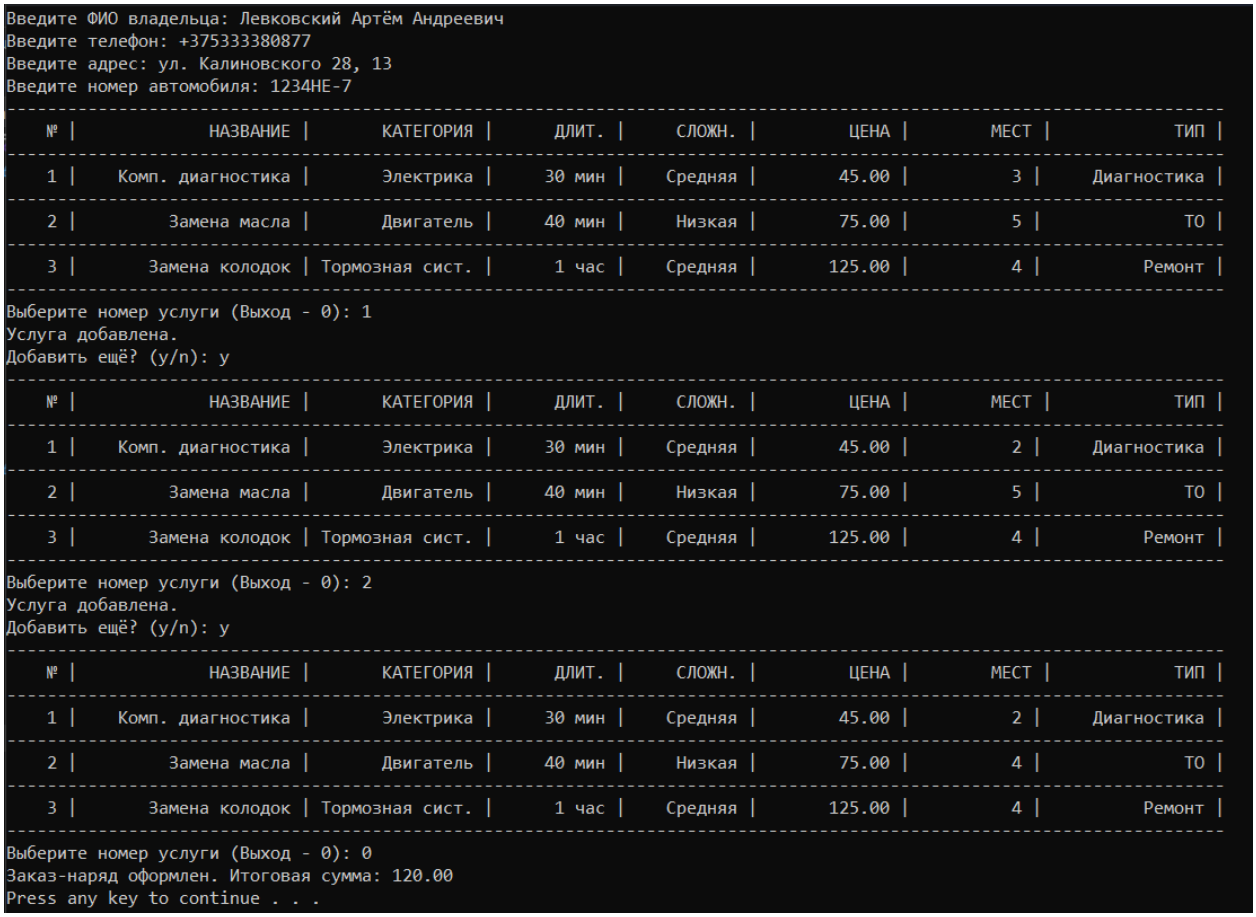


Рисунок 2 – Пример работы программы

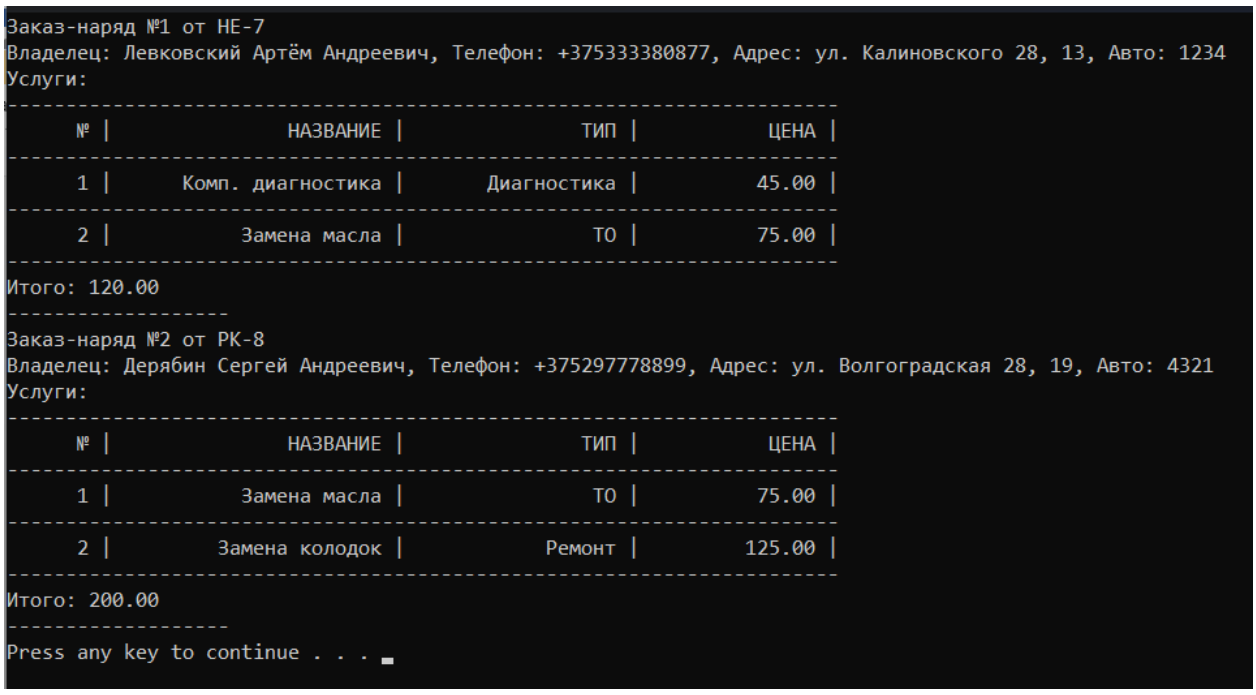


Рисунок 3 – Пример работы программы

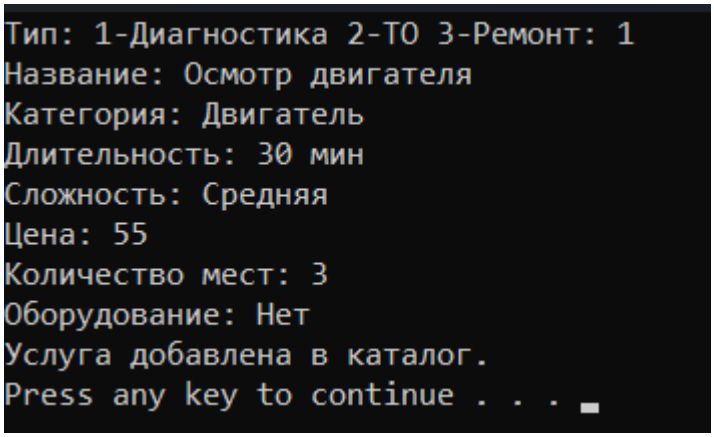


Рисунок 4 – Пример работы программы

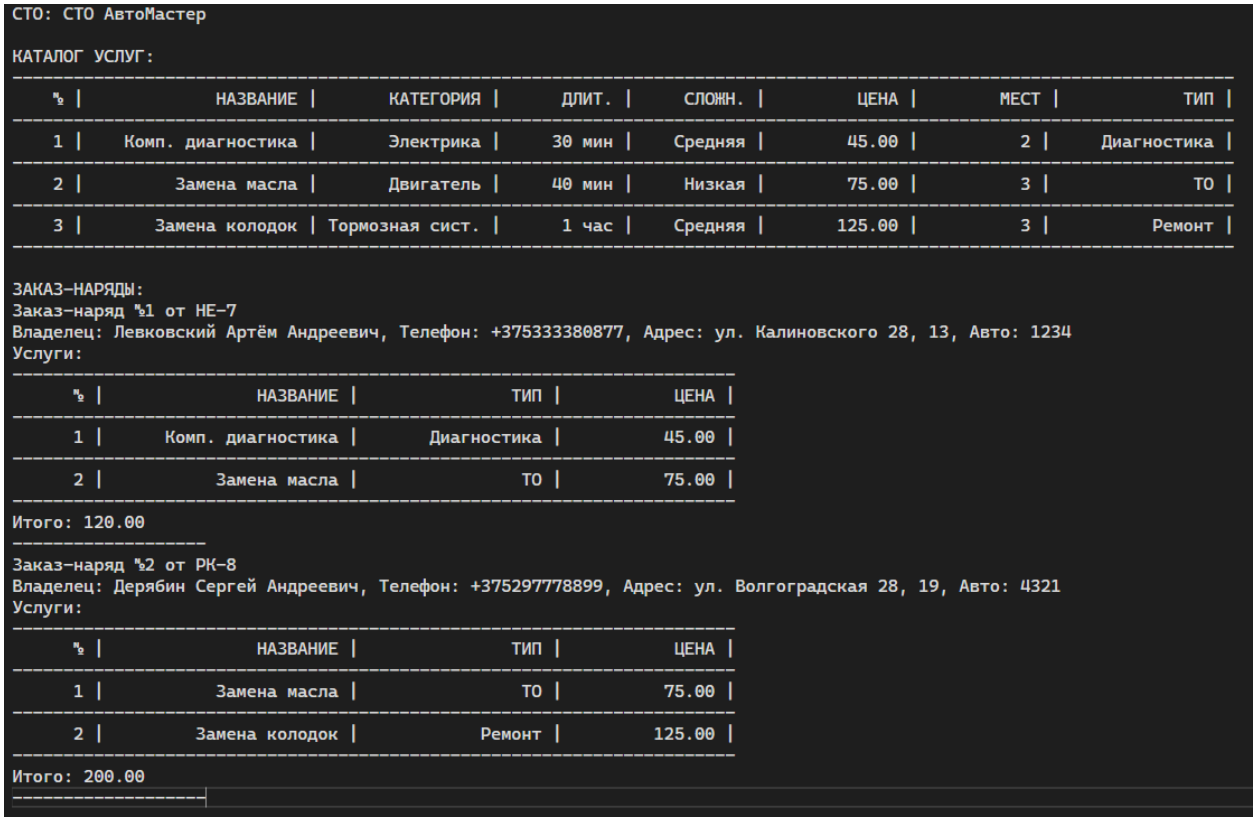


Рисунок 5 – Пример вывода в файл

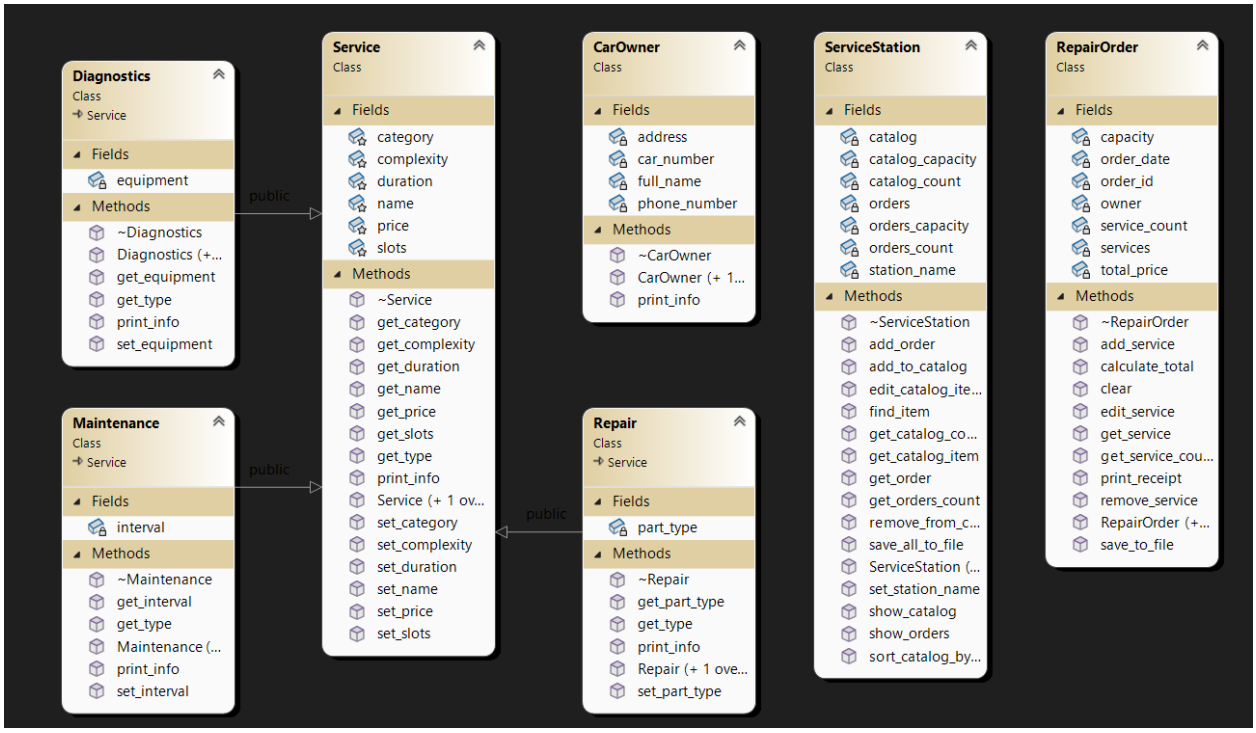


Рисунок 6 – UML диаграмма созданных классов

Выводы. В ходе выполнения лабораторной работы были изучены особенности работы с потоками ввода/вывода в C++. Были рассмотрены различные флаги и то, для чего они применяются. Также были изучены новые манипуляторы потока и разобран принцип их работы.